

subject	title	check
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	☐ ☐ ☐ ☐ ☐
		date

2026.06.01

Ch08 데이터베이스

핵심 260601 수업은 제공된 `Ch08\_데이터베이스.pptx` 자료 흐름을 그대로 따라가며, 데이터베이스 기본 개념, SQLite, Python `sqlite3`, PyQt5 기반 DB 관리 GUI까지 다뤘다.

확인 수업자료 원본 파일은 `C:/Users/ai/Desktop/공부/Ch08\_데이터베이스.pptx`이고, 실제 작성 코드는 `D:/Study/PYTHON/260601.py`이다.

데이터베이스 기본 개념

DBMS	데이터베이스를 관리하는 시스템 또는 소프트웨어
데이터베이스	여러 테이블이 저장되는 저장소
테이블	데이터가 행과 열로 정리된 표
열	각 데이터 항목의 이름과 데이터 형식을 가진 구조
행	실제 저장되는 한 건의 데이터
SQL	DBMS가 알아듣는 데이터베이스 조작 언어
RDBMS	관계형 데이터베이스 관리 시스템. MySQL, SQLite, Oracle, SQL Server 등이 해당

SQLite 구축 흐름

설치	SQLite 다운로드 페이지에서 Windows용 sqlite-tools 압축 파일을 내려받고 압축 해제
실행	`sqlite3.exe` 실행 후 `sqlite>` 프롬프트에서 명령 입력
데이터베이스 생성	`open 데이터베이스이름` 명령으로 기존 DB를 열거나 새 DB 생성
테이블 생성	`CREATE TABLE` SQL로 열 이름과 자료형 지정
데이터 입력	`INSERT INTO` SQL로 행 데이터 추가
데이터 조회	`SELECT`, `WHERE`, `ORDER BY`로 조건 조회와 정렬

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

Python sqlite3 처리 순서

연결	<code>`sqlite3.connect("DB 이름")`</code>
커서 생성	<code>`con.cursor()`</code>
SQL 실행	<code>`cur.execute(SQL문)`</code>
저장	<code>`con.commit()`</code>
닫기	<code>`con.close()`</code>
조회 반복	<code>`cur.fetchone()`</code> 으로 한 행씩 가져오고 <code>`None`</code> 이면 종료

PPT 슬라이드 원본 흐름

- 슬라이드 02 - Section 02 데이터베이스 기본 / 데이터베이스 개념 / 데이터베이스를 익히려면 데이터베이스 소프트웨어를 설치해야 함 / 이 소프트웨어를 DBMS(DataBase Management System 또는 DataBase Management Software) 라고 함 / DBMS 는 데이터베이스를 관리해 주는 시스템 또는 소프트웨어 / 데이터베이스 개념 / 대량의 데이터가 발생하는 현대 사회에서 파일 처리로는 한계가 있음 / 데이터베이스는 이를 보완하는 방법
- 슬라이드 03 - Section 02 데이터베이스 기본 / 데이터베이스 종류 / 계층형 (Hierarchical), 망형 (Network), 관계형 (Relational), 객체지향형 (Object -Oriented), 객체관계형 (Object-Relational) / 일반적으로 관계형 DBMS 를 가장 많이 사용 / 오라클 , SQL 서버 , 액세스 , MySQL, SQLite 등은 모두 관계형 DBMS, 즉 RDBMS 에 속함
- 슬라이드 04 - Section 02 데이터베이스 기본 / 데이터베이스 관련 용어 / 데이터베이스를 구축하려면 데이터베이스 모델링이라는 작업이 필요함 / 데이터베이스 모델링은 현실에서 사용되는 데이터를 DBMS 안에 어떻게 옮겨 놓을지 결정하는 과정
- 슬라이드 05 - Section 02 데이터베이스 기본 / 데이터베이스 관련 용어 / 데이터 : 단편적인 정보 / 테이블 : 데이터가 표 형태로 표현된 것 / 데이터베이스 : 테이블이 저장되는 저장소로 , 주로 원통 모양으로 표현 / DBMS: 데이터베이스를 관리하는 시스템 또는 소프트웨어를 의미 / 열 : 각 테이블은 1 개 이상의 열로 구성 / 열 이름 : 각 열을 구분하는 이름으로 , 각 테이블 안에서 중복되지 않아야 함 / 데이터 형식 : 열의 데이터 형식으로 테이블을 생성할 때 열 이름과 함께 지정 / 행 : 실질적인 데이터 / SQL(Structured Query Language): DBMS 가 알아듣는 언어
- 슬라이드 06 - Section 03 데이터베이스 구축 / DBMS 설치와 실행
- 슬라이드 07 - Section 03 데이터베이스 구축 / DBMS 설치와 실행 / SQLite 설치 페이지 (<https://www.sqlite.org/download.html>) 에 접속 / 'Precompiled Binaries for Windows' 항목에서 sqlite-tools-win32-x86- 버전 .zip 파일을 클릭하여 다운로드 / 다운로드한 파일의 압축을 풀고 , 폴더명을 sqlite 로 변경
- 슬라이드 08 - Section 03 데이터베이스 구축 / DBMS 설치와 실행 / 파일 탐색기에서 sqlite3.exe 파일을 더블클릭해 실행하면 명령 프롬프트창이 열리면서 sqlite > 로 표시됨
- 슬라이드 09 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / SQLite 에서 데이터베이스를 완성하는 것
- 슬라이드 10 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / 데이터베이스 생성 / 데이터베이스를 생성하거나 열려면 '.open 데이터베이스이름' 명령어를 실행 / 데이터베이스가 있다면 열어 주고 , 없다면 새로 생성하는 명령어 / 다음 명령어를 실행하면 [그림 8-3] 의 네이버 데이터베이스 (naverDB) 가 생성됨
- 슬라이드 11 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / 테이블 생성 / 테이블을 생성하는 SQL 문의 형식 / 앞서 계획한 [그림 8-3] 의 회원 테이블을 생성하고 확인하는 코드

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

- 슬라이드 12 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / 데이터 입력 / 행 데이터를 입력하는 SQL 문의 형식 / [그림 8-3] 의 회원 테이블에 행 4 개를 입력하는 SQL 문
- 슬라이드 13 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / 데이터 조회 및 활용 / 데이터를 조회 및 활용하는 SQL 문은 SELECT 로 일반적인 형식은 다음과 같음 / SELECT 문을 WHERE 조건과 함께 사용할 수도 있음
- 슬라이드 14 - Section 03 데이터베이스 구축 / [프로그램 1] 완성 / 데이터 조회 및 활용 / 1990 년 이전에 태어난 사람의 아이디와 출생연도를 확인 / 아이디가 ' park '인 사람의 모든 정보를 조회 *는 모든 열을 의미 / 조회한 결과를 정렬하기 위해 ORDER BY 문을 사용 / 작업이 모두 끝나 SQLite 를 종료
- 슬라이드 15 - Section 03 데이터베이스 구축 / [프로그램 1] 완성
- 슬라이드 16 - Section 04 데이터 입력과 조회 / 데이터 입력 순서
- 슬라이드 17 - Section 04 데이터 입력과 조회 / 데이터 입력 순서 / 데이터베이스 연결 / SQLite 를 사용하려면 먼저 관련 모듈인 sqlite3 를 임포트한 후 sqlite3.connect("DB 이름") 으로 데이터베이스와 연결 / 커서 생성 / 커서 (Cursor): 데이터베이스에 SQL 문을 실행하거나 실행된 결과를 돌려받는 통로
- 슬라이드 18 - Section 04 데이터 입력과 조회 / 데이터 입력 순서 / 테이블 만들기 / 테이블을 만드는 SQL 문을 커서이름 .execute() 함수의 매개변수로 넘겨주면 SQL 문이 데이터베이스에 실행됨
- 슬라이드 19 - Section 04 데이터 입력과 조회 / 데이터 입력 순서 / 데이터 입력 / 데이터는 필요한 만큼 반복해서 입력함
- 슬라이드 20 - Section 04 데이터 입력과 조회 / 데이터 입력 순서 / 입력한 데이터 저장 / 입력한 데이터 4 건은 아직 데이터베이스에 완전히 저장한 것이 아닌 임시로 저장된 상태 / 이를 확실하게 저장하는 것을 커밋 (Commit) 이라고 함 / 데이터베이스 닫기 / 데이터베이스를 모두 사용했다면 1 번에서 연결한 데이터베이스를 닫아야 함
- 슬라이드 21 - Section 04 데이터 입력과 조회 / 데이터 입력 프로그램 구현 / 사용자가 반복해서 데이터를 입력하는 코드 / 앞서 생성한 naverDB 의 userTable 에 Enter 를 입력하기 전까지 반복해서 한 행씩 데이터를 입력함
- 슬라이드 22 - Section 04 데이터 입력과 조회 / 데이터 입력 프로그램 구현 / Code08-01.py / 01 / 02 / 03 / 04 / 05 / 06 / 07 / 08 / 09 / 10 / 11 / 12 / 13 / 14 / 15 / 16 / 17 / 18 / 19 / 20 / 21 / 22 / import sqlite3 / ## 변수 선언 부분 ## / con, cur = None, None / data1, data2, data3, data4 = "", "", "", "" / sql = "" / ## 메인 코드 부분 ## / con = sqlite3.connect("C:/CookAnalysis/naverDB") # DB 가 저장된 폴더까지 지정 / cur = con.cursor() / while (True) : / data1 = input(" 사용자 ID ==> ") / if data1 == "" : / break / data2 = input(" 사용자이름 ==> ") / data3 = input(" 이메일 ==> ") / data4 = input(" 출생연도 ==> ") / sql = "INSERT INTO userTable VALUES(" + data1 + "," + data2 + "," + data3 + "," + data4 + ")" / cur.execute(sql) / con.commit() / con.close()
- 슬라이드 23 - Section 04 데이터 입력과 조회 / 데이터 입력 프로그램 구현
- 슬라이드 24 - Section 04 데이터 입력과 조회 / 데이터 조회 순서 / 파이썬에서 데이터를 조회하려면 아래 단계를 거쳐야 함
- 슬라이드 25 - Section 04 데이터 입력과 조회 / 데이터 조회 프로그램 구현 / Code08-02.py / 01 / 02 / 03 / 04 / 05 / 06 / 07 / 08 / 09 / 10 / 11 / 12 / 13 / 14 / 15 / 16 / 17 / 18 / 19 / 20 / import sqlite3 / ## 변수 선언 부분 ## / con, cur = None, None / data1, data2, data3, data4 = "", "", "", "" / row = None / ## 메인 코드 부분 ## / con = sqlite3.connect("C:/CookPython/naverDB") / cur = con.cursor() / cur.execute ("SELECT * FROM userTable ") / print(" 사용자 ID 사용자이름 이메일 출생연도 ") / print("-----") / while (True) : / row = cur.fetchone () / if row == None : / break / 21 / 22 / 23 / 24 / 25 / 26 / 27 / data1 = row[0] / data2 = row[1] / data3 = row[2] / data4 = row[3] / print("%5s %15s %20s %d" % (data1, data2, data3, data4)) / con.close ()
- 슬라이드 26 - Section 04 데이터 입력과 조회 / 데이터 조회 프로그램 구현
- 슬라이드 27 - Section 04 데이터 입력과 조회 / [프로그램 2] 완성 / GUI 화면에서 데이터를 입력하고 수정할 수 있도록 함 / Code08-03.py / 01 / 02 / 03 / 04 / 05 / 06 / 07 / 08 / 09 / 10 / 11 / 12 / 13 / 14 / 15 / 16 / import sqlite3 / from tkinter import * / from tkinter import messagebox / ## 함수 선언 부분 ## / def insertData () : / con,

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```
cur = None, None / data1, data2, data3, data4 = "", "", "", "" / sql = "" / con =
sqlite3.connect("C:/CookAnalysis/naverDB") # DB 가 저장된 폴더까지 지정 / cur = con.cursor () / data1 = edt1.get();
data2 = edt2.get(); data3 = edt3.get(); data4 = edt4.get() / try : / sql = "INSERT INTO userTable VALUES(" + data1
+ "," + data2 + " ,"
```

- 슬라이드 28 - Section 04 데이터 입력과 조회 / [프로그램 2] 완성 / Code08-03.py / 17 / 18 / 19 / 20 / 21 / 22 / 23 / 24 / 25 / 26 / 27 / 28 / 29 / 30 / 31 / 32 / 33 / 34 / "" + data3 + "," + data4 + ")" / cur.execute (sql) / except : / messagebox.showerror (' 오류 ' , ' 데이터 입력 오류가 발생함 ') / else : / messagebox.showinfo (' 성공 ' , ' 데이터 입력 성공 ') / con.commit () / con.close () / def selectData () : / strData1, strData2, strData3, strData4 = [], [], [], [] / con = sqlite3.connect("C:/CookAnalysis/naverDB") # DB 가 저장된 폴더까지 지정 / cur = con.cursor () / cur.execute ("SELECT * FROM userTable ") / strData1.append(" 사용자 ID"); strData2.append(" 사용자이름 ") / strData3.append(" 이메일 "); strData4.append(" 출생연도 ") / strData1.append("-----"); strData2.append("-----") / strData3.append("-----"); strData4.append("-----")
- 슬라이드 29 - Section 04 데이터 입력과 조회 / [프로그램 2] 완성 / Code08-03.py / 35 / 36 / 37 / 38 / 39 / 40 / 41 / 42 / 43 / 44 / 45 / 46 / 47 / 48 / 49 / 50 / 51 / 52 / 53 / 54 / 55 / while (True) : / row = cur.fetchone () / if row == None : / break / strData1.append(row[0]); strData2.append(row[1]) / strData3.append(row[2]); strData4.append(row[3]) / listData1.delete(0, listData1.size() - 1); listData2.delete(0, listData2.size() - 1) / listData3.delete(0, listData3.size() - 1); listData4.delete(0, listData4.size() - 1) / for item1, item2, item3, item4 in zip(strData1, strData2, strData3, strData4) : / listData1.insert(END, item1); listData2.insert(END, item2) / listData3.insert(END, item3); listData4.insert(END, item4) / con.close () / ## 메인 코드 부분 ## / window = Tk() / window.geometry ("600x300") / window.title ("GUI 데이터 입력 ") / edtFrame = Frame(window); / edtFrame.pack ()
- 슬라이드 30 - Section 04 데이터 입력과 조회 / [프로그램 2] 완성 / Code08-03.py / 56 / 57 / 58 / 59 / 60 / 61 / 62 / 63 / 64 / 65 / 66 / 67 / 68 / 69 / 70 / 71 / 72 / 73 / 74 / 75 / 76 / 77 / listFrame = Frame(window) / listFrame.pack(side = BOTTOM, fill = BOTH, expand = 1) / edt1 = Entry(edtFrame , width = 10); edt1.pack(side = LEFT, padx = 10, pady = 10) / edt2 = Entry(edtFrame , width = 10); edt2.pack(side = LEFT, padx = 10, pady = 10) / edt3 = Entry(edtFrame , width = 10); edt3.pack(side = LEFT, padx = 10, pady = 10) / edt4 = Entry(edtFrame , width = 10); edt4.pack(side = LEFT, padx = 10, pady = 10) / btnInsert = Button(edtFrame , text = " 입력 ", command = insertData) / btnInsert.pack (side = LEFT, padx = 10, pady = 10) / btnSelect = Button(edtFrame , text = " 조회 ", command = selectData) / btnSelect.pack (side = LEFT, padx = 10, pady = 10) / listData1 = Listbox (listFrame , bg = 'yellow'); / listData1.pack(side = LEFT, fill = BOTH, expand = 1) / listData2 = Listbox (listFrame , bg = 'yellow') / listData2.pack(side = LEFT, fill = BOTH, expand = 1) / listData3 = Listbox (listFrame , bg = 'yellow') / listData3.pack(side = LEFT, fill = BOTH, expand = 1) / listData4 = Listbox (listFrame , bg = 'yellow') / listData4.pack(side = LEFT, fill = BOTH, expand = 1) / window.mainloop ()
- 슬라이드 31 - Section 04 데이터 입력과 조회 / [프로그램 2] 완성

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

실제 수업 코드: SQLite Manager - PyQt5

주석

PPT의 SQLite 입력/조회 예제를 확장해서, PyQt5 GUI에서 테이블 목록 조회, 테이블 생성/삭제, 행 추가/삭제, 셀 수정 저장까지 처리하는 프로그램이다.

```
import os
import sqlite3
import sys

from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget, QListWidget, QTableWidget,
    QTableWidgetItem, QPushButton, QLabel, QVBoxLayout, QHBoxLayout,
    QSplitter, QMessageBox, QDialog, QLineEdit, QHeaderView
)
from PyQt5.QtCore import Qt

DB_PATH = "./files/DB/naverDB"

class CreateTableDialog(QDialog):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("테이블 생성")
        self.resize(650, 420)

        self.table_name = QLineEdit()
        self.table_name.setPlaceholderText("테이블명 입력")

        self.column_table = QTableWidget(1, 3)
        self.column_table.setHorizontalHeaderLabels(["컬럼명", "자료형", "PK"])
        self.column_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)

        self.column_table.setItem(0, 0, QTableWidgetItem(""))
        self.column_table.setItem(0, 1, QTableWidgetItem("CHAR(15)"))
        self.column_table.setItem(0, 2, QTableWidgetItem("NO"))

        add_btn = QPushButton("컬럼 추가")
        del_btn = QPushButton("선택 컬럼 삭제")
        ok_btn = QPushButton("생성")
        cancel_btn = QPushButton("취소")

        add_btn.clicked.connect(self.add_column)
        del_btn.clicked.connect(self.delete_column)
        ok_btn.clicked.connect(self.accept)
        cancel_btn.clicked.connect(self.reject)
```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

layout = QVBoxLayout()
layout.addWidget(QLabel("테이블명"))
layout.addWidget(self.table_name)
layout.addWidget(QLabel("컬럼 정보"))
layout.addWidget(self.column_table)

btns = QHBoxLayout()
btns.addWidget(add_btn)
btns.addWidget(del_btn)
btns.addStretch()
btns.addWidget(ok_btn)
btns.addWidget(cancel_btn)

layout.addLayout(btns)
self.setLayout(layout)

def add_column(self):
    row = self.column_table.rowCount()
    self.column_table.insertRow(row)
    self.column_table.setItem(row, 0, QTableWidgetItem(""))
    self.column_table.setItem(row, 1, QTableWidgetItem("CHAR(15)"))
    self.column_table.setItem(row, 2, QTableWidgetItem("NO"))

def delete_column(self):
    row = self.column_table.currentRow()
    if row >= 0:
        self.column_table.removeRow(row)

def get_data(self):
    table_name = self.table_name.text().strip()
    columns = []

    for r in range(self.column_table.rowCount()):
        name_item = self.column_table.item(r, 0)
        type_item = self.column_table.item(r, 1)
        pk_item = self.column_table.item(r, 2)

        name = name_item.text().strip() if name_item else ""
        col_type = type_item.text().strip() if type_item else "TEXT"
        pk = pk_item.text().strip().upper() if pk_item else "NO"

    if name:
        columns.append((name, col_type, pk))

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```
return table_name, columns
```

```
class SQLiteManager(QMainWindow):
```

```
def __init__(self):
```

```
super().__init__()
```

```
self.setWindowTitle("SQLite Manager - PyQt5")
```

```
self.resize(1200, 750)
```

```
os.makedirs("./files/DB", exist_ok=True)
```

```
self.con = sqlite3.connect(DB_PATH)
```

```
self.cur = self.con.cursor()
```

```
self.current_table = None
```

```
self.columns = []
```

```
self.rowids = []
```

```
self.loading = False
```

```
self.init_ui()
```

```
self.load_tables()
```

```
def init_ui(self):
```

```
self.setStyleSheet("""
```

```
QMainWindow {
```

```
background-color: #f3f4f6;
```

```
}
```

```
QListWidget {
```

```
background: white;
```

```
border: 1px solid #d1d5db;
```

```
border-radius: 8px;
```

```
padding: 6px;
```

```
font-size: 14px;
```

```
}
```

```
QListWidget::item {
```

```
padding: 8px;
```

```
border-radius: 6px;
```

```
}
```

```
QListWidget::item:selected {
```

```
background-color: #2563eb;
```

```
color: white;
```

```
}
```

```
QTableWidget {
```

```
background: white;
```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

border: 1px solid #d1d5db;
border-radius: 8px;
gridline-color: #e5e7eb;
font-size: 14px;
selection-background-color: #bfbdbf;
}

QHeaderView::section {
background-color: #e5e7eb;
padding: 8px;
border: none;
font-weight: bold;
}

QPushButton {
background-color: #2563eb;
color: white;
border: none;
border-radius: 8px;
padding: 8px 14px;
font-size: 13px;
}

QPushButton:hover {
background-color: #1d4ed8;
}

QLabel {
font-size: 14px;
}

QLineEdit {
padding: 8px;
border: 1px solid #d1d5db;
border-radius: 6px;
font-size: 14px;
}

"""

splitter = QSplitter(Qt.Horizontal)

left = QWidget()
left_layout = QVBoxLayout()

title = QLabel("테이블 목록")
title.setStyleSheet("font-size: 20px; font-weight: bold;")

self.table_list = QListWidget()
self.table_list.itemDoubleClicked.connect(self.open_table)

```


subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

refresh_btn = QPushButton("새로고침")
create_btn = QPushButton("테이블 생성")
drop_btn = QPushButton("테이블 삭제")

refresh_btn.clicked.connect(self.load_tables)
create_btn.clicked.connect(self.create_table)
drop_btn.clicked.connect(self.drop_table)

schema_title = QLabel("컬럼 정보")
schema_title.setStyleSheet("font-size: 17px; font-weight: bold; margin-top: 12px;")

self.schema_list = QListWidget()

left_layout.addWidget(title)
left_layout.addWidget(self.table_list)
left_layout.addWidget(refresh_btn)
left_layout.addWidget(create_btn)
left_layout.addWidget(drop_btn)
left_layout.addWidget(schema_title)
left_layout.addWidget(self.schema_list)

left.setLayout(left_layout)

right = QWidget()
right_layout = QVBoxLayout()

self.table_title = QLabel("테이블을 선택하세요")
self.table_title.setStyleSheet("font-size: 24px; font-weight: bold;")

btn_layout = QHBoxLayout()

add_row_btn = QPushButton("빈 행 추가")
delete_row_btn = QPushButton("선택 행 삭제")
reload_btn = QPushButton("다시 불러오기")

add_row_btn.clicked.connect(self.add_empty_row)
delete_row_btn.clicked.connect(self.delete_selected_rows)
reload_btn.clicked.connect(self.load_data)

btn_layout.addWidget(add_row_btn)
btn_layout.addWidget(delete_row_btn)
btn_layout.addWidget(reload_btn)
btn_layout.addStretch()

self.data_table = QTableWidget()

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

self.data_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
self.data_table.itemChanged.connect(self.cell_changed)

right_layout.addWidget(self.table_title)
right_layout.addLayout(btn_layout)
right_layout.addWidget(self.data_table)

right.setLayout(right_layout)

splitter.addWidget(left)
splitter.addWidget(right)
splitter.setSizes([280, 920])

self.setCentralWidget(splitter)

def load_tables(self):
    self.table_list.clear()

    self.cur.execute("SELECT name FROM sqlite_master WHERE type='table'")
    tables = [row[0] for row in self.cur.fetchall()]

    for table in tables:
        self.table_list.addItem(table)

    def open_table(self, item):
        self.current_table = item.text()
        self.table_title.setText(self.current_table)
        self.load_schema()
        self.load_data()

    def get_selected_table_name(self):
        item = self.table_list.currentItem()
        if item:
            return item.text()
        return self.current_table

    def load_schema(self):
        self.schema_list.clear()

        if not self.current_table:
            return

        self.cur.execute(f'PRAGMA table_info("{self.current_table}")')
        info = self.cur.fetchall()

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

for col in info:
    name = col[1]
    col_type = col[2]
    pk = " PK" if col[5] == 1 else ""
    self.schema_list.addItem(f"{name} {col_type}{pk}")

def load_data(self):
    if not self.current_table:
        return

    self.loading = True

    self.cur.execute(f'PRAGMA table_info("{self.current_table}")')
    info = self.cur.fetchall()
    self.columns = [col[1] for col in info]

    self.cur.execute(f'SELECT rowid, * FROM "{self.current_table}"')
    rows = self.cur.fetchall()

    self.rowids = [row[0] for row in rows]
    data = [row[1:] for row in rows]

    self.data_table.clear()
    self.data_table.setRowCount(len(data))
    self.data_table.setColumnCount(len(self.columns))
    self.data_table.setHorizontalHeaderLabels(self.columns)

    for r, row in enumerate(data):
        for c, value in enumerate(row):
            item = QTableWidgetItem("" if value is None else str(value))
            item.setTextAlignment(Qt.AlignCenter)
            self.data_table.setItem(r, c, item)

    self.loading = False

def cell_changed(self, item):
    if self.loading or not self.current_table:
        return

    row = item.row()
    col = item.column()

    if row >= len(self.rowids):
        return

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

rowid = self.rowids[row]
col_name = self.columns[col]
new_value = item.text()

self.cur.execute(
f'UPDATE "{self.current_table}" SET "{col_name}"=? WHERE rowid=?',
(new_value, rowid)
)
self.con.commit()

def add_empty_row(self):
if not self.current_table:
QMessageBox.warning(self, "경고", "테이블을 먼저 선택하세요.")
return

values = [""] * len(self.columns)
col_names = ", ".join([f"{c}" for c in self.columns])
placeholders = ", ".join(["?"] * len(self.columns))

self.cur.execute(
f'INSERT INTO "{self.current_table}" ({col_names}) VALUES ({placeholders})',
values
)
self.con.commit()
self.load_data()

def delete_selected_rows(self):
if not self.current_table:
return

rows = sorted(
set(index.row() for index in self.data_table.selectedIndexes()),
reverse=True
)

if not rows:
QMessageBox.warning(self, "경고", "삭제할 행을 선택하세요.")
return

reply = QMessageBox.question(
self,
"삭제 확인",
"선택한 행을 삭제할까요?",
QMessageBox.Yes | QMessageBox.No
)

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

if reply != QMessageBox.Yes:
    return

for row in rows:
    rowid = self.rowids[row]
    self.cur.execute(
        f'DELETE FROM "{self.current_table}" WHERE rowid=?',
        (rowid,)
    )

    self.con.commit()
    self.load_data()

def create_table(self):
    dialog = CreateTableDialog()

    if dialog.exec_() != QDialog.Accepted:
        return

    table_name, columns = dialog.get_data()

    if not table_name:
        QMessageBox.warning(self, "경고", "테이블명을 입력하세요.")
        return

    if not columns:
        QMessageBox.warning(self, "경고", "컬럼을 하나 이상 입력하세요.")
        return

    pk_count = 0
    sql_columns = []

    for col_name, col_type, pk in columns:
        if pk == "YES":
            pk_count += 1
            sql_columns.append(f'"{col_name}" {col_type} PRIMARY KEY')
        else:
            sql_columns.append(f'"{col_name}" {col_type}')

    if pk_count > 1:
        QMessageBox.warning(self, "경고", "PK는 하나만 설정하세요.")
        return

    sql = f'CREATE TABLE IF NOT EXISTS "{table_name}" ({", ".join(sql_columns)})'

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

try:
    self.cur.execute(sql)
    self.con.commit()
    self.load_tables()
    QMessageBox.information(self, "완료", f"{table_name} 테이블 생성 완료")
except Exception as e:
    QMessageBox.critical(self, "오류", str(e))

def drop_table(self):
    table_name = self.get_selected_table_name()

    if not table_name:
        QMessageBox.warning(self, "경고", "삭제할 테이블을 선택하세요.")
        return

    reply = QMessageBox.question(
        self,
        "테이블 삭제",
        f"{table_name} 테이블을 삭제할까요?",
        QMessageBox.Yes | QMessageBox.No
    )

    if reply != QMessageBox.Yes:
        return

    self.cur.execute(f'DROP TABLE IF EXISTS "{table_name}"')
    self.con.commit()

    self.current_table = None
    self.table_title.setText("테이블을 선택하세요")
    self.schema_list.clear()
    self.data_table.clear()
    self.data_table.setRowCount(0)
    self.data_table.setColumnCount(0)

    self.load_tables()

def closeEvent(self, event):
    self.con.close()
    event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)

```

subject	title	check ☐ ☐ ☐ ☐ ☐
노시영교수님	노시영교수님 - 데이터베이스 SQLite 개념정리	date

```

window = SQLiteManager()
window.show()
sys.exit(app.exec_())

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

추가 SQL 실습 파일

주석 `D:/Study/SQL/test.sql`에 남아 있던 MySQL/MariaDB 테이블 생성과 조회/입력 SQL이다.

```

CREATE TABLE `my_community`.`user` (
  `user_id` VARCHAR(50) NOT NULL,
  `age` INT UNSIGNED NULL,
  `nickname` VARCHAR(200) NULL,
  `email` VARCHAR(200) NULL,
  `password` VARCHAR(300) NULL,
  `created_at` DATETIME NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`user_id`)); -- 테이블생성 명령

show databases; -- 데이터베이스 목록 조회
use my_community; -- 사용할 데이터베이스 스키마 선택
show tables; -- 선택된 스키마의 테이블 목록 조회
select * from user; -- user 테이블의 전체 데이터 조회

-- 데이터 입력
insert into user (user_id, age, nickname) values ('abc', 28, '테스트');
INSERT INTO `my_community`.`user` (`user_id`, `age`, `nickname`) VALUES ('wow', '55', '개쩌네');

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.